



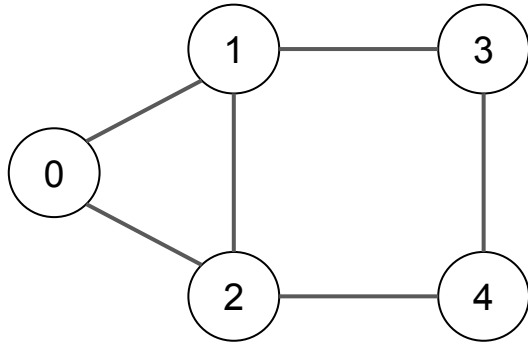
Breadth First Search

CSE 2104: Data Structure and Algorithms-I Lab

Breadth First Search (BFS) for a Graph Algorithm:

1. **Initialization:** Enqueue the starting node into a queue and mark it as visited.
2. **Exploration:** While the queue is not empty:
 - Dequeue a node from the queue and visit it (e.g., print its value).
 - For each unvisited neighbor of the dequeued node:
 - Enqueue the neighbor into the queue.
 - Mark the neighbor as visited.
3. **Termination:** Repeat step 2 until the queue is empty.

BFS Illustration



Visited

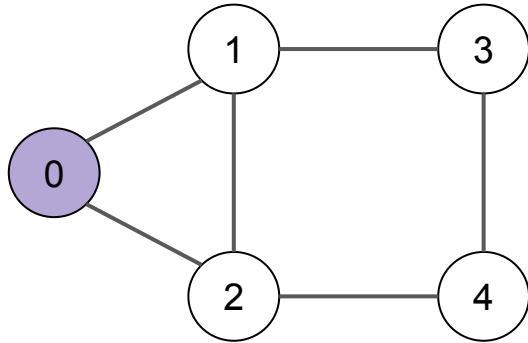


Queue



Step1: Initially queue and visited arrays are empty.

BFS Illustration



Visited

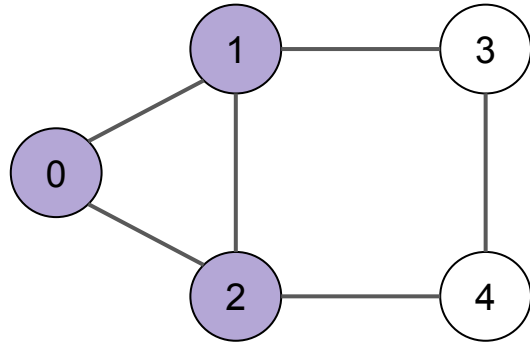
0				
---	--	--	--	--

Queue

0				
---	--	--	--	--

Step2: Push node 0 into queue and mark it visited.

BFS Illustration



Visited

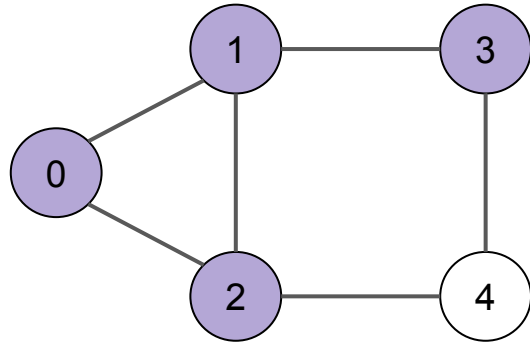
0	1	2		
---	---	---	--	--

Queue

1	2			
---	---	--	--	--

Step 3: Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.

BFS Illustration



Visited

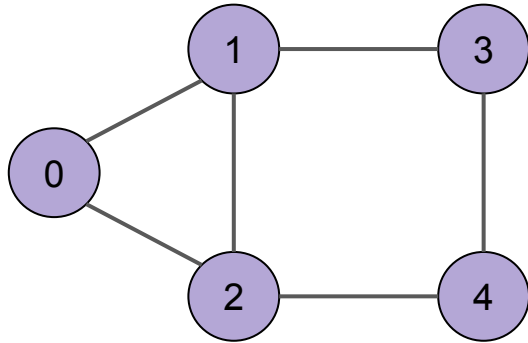
0	1	2	3	
---	---	---	---	--

Queue

2	3			
---	---	--	--	--

Step 4: Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.

BFS Illustration



Visited

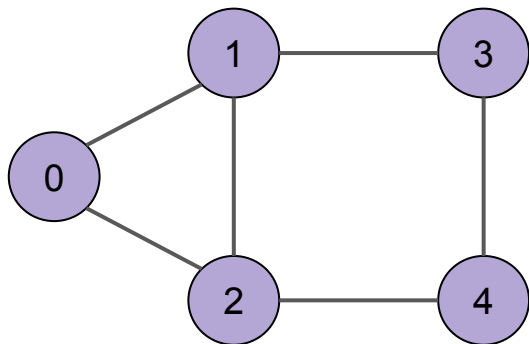
0	1	2	3	4
---	---	---	---	---

Queue

3	4			
---	---	--	--	--

Step 5: Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.

BFS Illustration



Visited

0	1	2	3	4
---	---	---	---	---

Queue

4				
---	--	--	--	--

Step 6: Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.

As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.

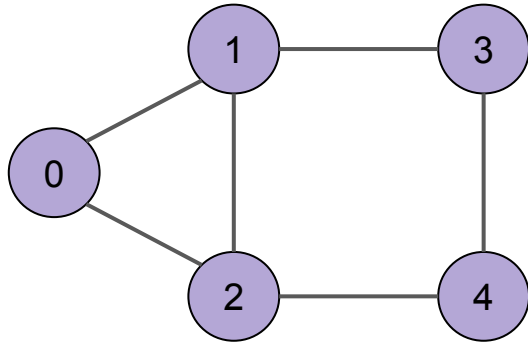
BFS Illustration



Steps 7: Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.

As we can see that every neighbours of node 4 are visited, so move to the next node that is in the front of the queue.

BFS Illustration



Visited

0	1	2	3	4
---	---	---	---	---

Queue

--	--	--	--	--

Termination: The Queue is now empty, So, we terminate the iteration.

Adjacency List Representation of Graph

```
int vertices = 5;  
vector<vector<int>> adjList(vertices);
```

Adjacency List:

```
Vertex 0 -> 1 2  
Vertex 1 -> 0 2 3  
Vertex 2 -> 0 1 4  
Vertex 3 -> 1 4  
Vertex 4 -> 2 3
```

adj-list-template.cpp

Adjacency List Representation of Graph

```
int vertices = 5;  
vector<vector<int>> adjList(vertices);  
  
addEdge(adjList, 0, 1);  
addEdge(adjList, 0, 2);  
addEdge(adjList, 1, 2);  
addEdge(adjList, 1, 3);  
addEdge(adjList, 2, 4);  
addEdge(adjList, 3, 4);
```

Adjacency List Representation of Graph

```
void addEdge(vector<vector<int>>& adjList, int u, int v)
{
    adjList[u].push_back(v);
    adjList[v].push_back(u);
}
```

Adjacency List Representation of Graph

```
void printAdjacencyList(const vector<vector<int>>& adjList) {  
    for (int i = 0; i < adjList.size(); ++i) {  
        cout << "Vertex " << i << " -> ";  
        for (int neighbor : adjList[i]) {  
            cout << neighbor << " ";  
        }  
        cout << endl;  
    }  
}
```

Adjacency List Representation of Graph

-

-

```
addEdge(adjList, 2, 4);
```

```
addEdge(adjList, 3, 4);
```

```
cout << "Adjacency List:" << endl;
```

```
printAdjacencyList(adjList);
```

Matrix Representation of Graph

```
int vertices = 5;  
  
vector<vector<int>> adjMatrix(vertices, vector<int>(vertices,  
0));  
  
addEdge(adjMatrix, 0, 1);  
addEdge(adjMatrix, 0, 2);  
addEdge(adjMatrix, 1, 2);  
addEdge(adjMatrix, 1, 3);  
addEdge(adjMatrix, 2, 4);  
addEdge(adjMatrix, 3, 4);
```

matrix-template.cpp

Matrix Representation of Graph

```
void addEdge(vector<vector<int>>& adjMatrix, int u, int v) {  
    adjMatrix[u][v] = 1;  
    adjMatrix[v][u] = 1; // For undirected graph  
}
```

Adjacency Matrix:

```
0 1 1 0 0  
1 0 1 1 0  
1 1 0 0 1  
0 1 0 0 1  
0 0 1 1 0
```

Matrix Representation of Graph

```
void printAdjacencyMatrix(const vector<vector<int>>& adjMatrix) {  
    for (int i = 0; i < adjMatrix.size(); ++i) {  
        for (int j = 0; j < adjMatrix[i].size(); ++j) {  
            cout << adjMatrix[i][j] << " ";  
        }  
        cout << endl;  
    }  
}
```

Matrix Representation of Graph

```
.  
.   
addEdge(adjMatrix, 2, 4);  
addEdge(adjMatrix, 3, 4);  
  
cout << "Adjacency Matrix:" << endl;  
printAdjacencyMatrix(adjMatrix);
```

BFS With Adjacency List

```
int vertices = 5;  
  
vector<vector<int> > adjList(vertices);
```

- creates a 2D vector
- `vector<int>` is a vector that stores integers,
- `vector<vector<int>>` represents a vector that stores vectors of integers
- specifies the number of rows (or outer vectors) that the `adjList` will contain

bfs-template.cpp

BFS With Adjacency List

```
int vertices = 5;  
  
vector<vector<int> > adjList(vertices);
```

Vertex 0 -> 1 2

Vertex 1 -> 2 3

Vertex 2 -> 4

Vertex 3 -> 4

Vertex 4 ->

BFS With Adjacency List

```
int vertices = 5;

vector<vector<int> > adjList(vertices);

addEdge(adjList, 0, 1);
addEdge(adjList, 0, 2);
addEdge(adjList, 1, 2);
addEdge(adjList, 1, 3);
addEdge(adjList, 2, 4);
addEdge(adjList, 3, 4);
```

BFS With Adjacency List

```
void addEdge(vector<vector<int> >& adjList, int u, int v){  
    adjList[u].push_back(v);  
}
```

BFS With Adjacency List

```
void printAdjacencyList(const vector<vector<int>>& adjList)
{
    for (int i = 0; i < adjList.size(); i++) {
        cout << "Vertex " << i << " -> ";
        for (int j = 0; j < adjList[i].size(); j++) {
            cout << adjList[i][j] << " ";
        }
        cout << endl;
    }
}
```

BFS With Adjacency List

```
addEdge(adjList, 0, 1);  
addEdge(adjList, 0, 2);  
addEdge(adjList, 1, 2);  
addEdge(adjList, 1, 3);  
addEdge(adjList, 2, 4);  
addEdge(adjList, 3, 4);  
  
cout << "Adjacency List:" << endl;  
printAdjacencyList(adjList);
```

BFS With Adjacency List

```
addEdge(adjList, 0, 1);  
addEdge(adjList, 0, 2);  
addEdge(adjList, 1, 2);  
addEdge(adjList, 1, 3);  
addEdge(adjList, 2, 4);  
addEdge(adjList, 3, 4);  
  
cout << "Adjacency List:" << endl;  
printAdjacencyList(adjList);
```

Adjacency List:

```
Vertex 0 -> 1 2  
Vertex 1 -> 2 3  
Vertex 2 -> 4  
Vertex 3 -> 4  
Vertex 4 ->
```

BFS With Adjacency List

```
cout << "Adjacency List:" << endl;
printAdjacencyList(adjList);

vector<bool> visited(vertices, false);

cout << "Breadth First Traversal starting from vertex "
      "0: ";
bfs(adjList, 0, visited);
```

BFS With Adjacency List

```
void bfs(vector<vector<int> >& adjList, int startNode,  
        vector<bool>& visited)
```

BFS With Adjacency List

```
queue<int> q;  
  
visited[startNode] = true;  
  
q.push(startNode);
```

BFS With Adjacency List

```
while (!q.empty()) {  
    int currentNode = q.front();  
    q.pop();  
    cout << currentNode << " ";  
  
    for (int neighbor : adjList[currentNode]) {  
        if (!visited[neighbor]) {  
            visited[neighbor] = true;  
            q.push(neighbor);  
        }  
    }  
}
```

BFS With Adjacency List

```
while (!q.empty()) {
    int currentNode = q.front();
    q.pop();
    cout << currentNode << " ";

    for (int j = 0; j < adjList[currentNode].size(); j++) {
        int neighbor = adjList[currentNode][j];

        if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}
```

BFS With Adjacency Matrix

```
int vertices = 5;

vector<vector<int>> adjMatrix(vertices, vector<int>(vertices,
0));

addEdge(adjMatrix, 0, 1);
addEdge(adjMatrix, 0, 2);
addEdge(adjMatrix, 1, 2);
addEdge(adjMatrix, 1, 3);
addEdge(adjMatrix, 2, 4);
addEdge(adjMatrix, 3, 4);
```

bfs-matrix-template.cpp

BFS With Adjacency Matrix

```
void addEdge(vector<vector<int>>& adjMatrix, int u, int v)
{
    adjMatrix[u][v] = 1;
    adjMatrix[v][u] = 1; // For undirected graph
}
```

Adjacency Matrix:

```
0 1 1 0 0
1 0 1 1 0
1 1 0 0 1
0 1 0 0 1
0 0 1 1 0
```

BFS With Adjacency Matrix

```
void printAdjacencyMatrix(const vector<vector<int>>& adjMatrix)
{
    for (int i = 0; i < adjMatrix.size(); i++) {
        for (int j = 0; j < adjMatrix[i].size(); j++) {
            int val = adjMatrix[i][j];
            cout << val << " ";
        }

        cout << endl;
    }
}
```

BFS With Adjacency Matrix

```
cout << "Adjacency Matrix:" << endl;  
    printAdjacencyMatrix(adjMatrix);  
  
    vector<bool> visited(vertices, false);  
  
    cout << "\nBreadth First Traversal starting from vertex 0: ";  
    bfs(adjMatrix, 0, visited);
```

BFS With Adjacency Matrix

```
void bfs(vector<vector<int>>& adjMatrix, int startNode,  
        vector<bool>& visited)
```

BFS With Adjacency Matrix

```
void bfs(vector<vector<int>>& adjMatrix, int startNode,  
        vector<bool>& visited)  
{  
    queue<int> q;  
  
    visited[startNode] = true;  
    q.push(startNode);  
    .  
    .  
    .
```

BFS With Adjacency Matrix

```
while (!q.empty()) {
    int currentNode = q.front();
    q.pop();
    cout << currentNode << " ";

    for (int neighbor = 0; neighbor < adjMatrix[currentNode].size();
neighbor++) {
        if (adjMatrix[currentNode][neighbor] && !visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}
```

Complexity Analysis of Breadth-First Search (BFS) Algorithm:

Time Complexity of BFS Algorithm: $O(V + E)$

- BFS explores all the vertices and edges in the graph. In the worst case, it visits every vertex and edge once. Therefore, the time complexity of BFS is $O(V + E)$, where V and E are the number of vertices and edges in the given graph.

Space Complexity of BFS Algorithm: $O(V)$

- BFS uses a queue to keep track of the vertices that need to be visited. In the worst case, the queue can contain all the vertices in the graph. Therefore, the space complexity of BFS is $O(V)$, where V and E are the number of vertices and edges in the given graph. The auxiliary space required by the algorithm grows linearly with the number of vertices in the graph.

BFS for disconnected graph

```
bfs-disconnected-graph.cpp
```

Applications of Breadth-First Search (BFS) Algorithm:

- Finding connected components
- Finding shortest path

```
bfs-connected-components.cpp  
bfs-shortest-path.cpp
```

List of files

- `matrix-template.cpp`
- `adj-list-template.cpp`
- `bfs-template.cpp`
- `bfs-matrix-template.cpp`
- `bfs-disconnected-graph.cpp`
- `bfs-connected-components.cpp`
- `bfs-shortest-path.cpp`